Oracle Study: Per-Token Sensitivity to KV-Cache Quantization

TLDR;

- **Goal:** find a cheap way to tell which tokens matter most, so the KV cache can keep those at full precision and store the rest in fewer bits.
- **The answer key:** I damaged one token at a time (stored it in 4 bits) and measured how much the model's predictions changed. That shows how sensitive each token really is.
- **The first token matters most.** Damaging it hurts about 15× more than a typical token. After the first ~50 tokens, position barely matters.
- **Best cheap signal: raw attention.** It matches the answer key best of the three signals (raw attention, normalized attention, key norm) I tested. Key norm barely beats random guessing.
- **Next:** use raw attention inside the cache, and check whether it actually keeps the model's quality up.

What I'm testing

Our project saves memory by storing tokens in the KV cache with fewer bits. But some tokens matter more than others. Squeezing an important token hurts the model a lot, and squeezing an unimportant one barely matters. So the cache needs to know which tokens are important.

Think of blurring one word in an essay and checking how much the meaning changes. The oracle does that for every token, one at a time. The result is an answer key: how sensitive each token really is.

The oracle is too slow for a real system, so a real system has to guess from cheap signals the model already produces. I tested three, each from a different paper:

- **Raw attention** (H2O): how much attention a token gets from the tokens after it. More attention probably means more important.
- **Normalized attention** (ZipCache): the same idea, but as an average, so tokens near the end aren't punished for having fewer tokens after them.
- **Key norm** (Devoto et al.): the size of the token's key vector. The paper found that smaller keys tend to get more attention.

Then I graded each signal against the answer key.

Key terms

Term	Meaning
KV cache	The model's memory of earlier tokens. Each token stores a key and a value in every layer.
Quantization	Storing numbers with fewer bits. It saves memory but adds rounding error.
INT4, FP16, FP32	4-bit, 16-bit and 32-bit numbers. The cache normally uses 16 bits.
Token j, position t	j is the token I damage. t is where I check the damage.
KL divergence	One number for how much a prediction changed. 0 means no change.
Sensitivity score	How much damaging one token hurts the next H predictions, on average.
Attention sink	A token, usually the first, that the model leans on no matter what it says.
Perplexity	How surprised the model is by real text. Lower is better.
Match score (Kendall)	How similar two rankings are. 1 means identical, 0 means no better than random.

What I ran it on

Choice	What I used	Why
Text	WikiText-2 (test set)	The standard text for this kind of test, so results compare with other papers.
Model	SmolLM-360M	Small and fast, but built like Llama-3-8B, our main model.
Inputs	20 pieces (”chunks”) of 512 tokens each	Enough to see clear patterns without taking forever.
Number format	FP32	Avoids rounding noise that could hide the tiny changes I'm measuring.
Damage level	INT4	It hurts quality a little but not a lot, which is where picking the right tokens matters.

I kept the setup small because the oracle reruns the model once for every token. Doubling the chunk length would cost at least 4× as much.

How the oracle works

1. **Clean run.** Run the model normally (FP32) on all 20 chunks, and save its prediction for the next word at every position.

2. **Damaged run.** Pick one token. Store only that token's keys and values in 4 bits, in every layer, and run the model again.
3. **Compare.** At every position, measure how much the prediction changed, using KL divergence.
4. **Repeat** for all 512 tokens.

KL divergence in plain words: the model's prediction is a list of probabilities for every possible next word. KL compares the clean list with the damaged list and gives one number: 0 if nothing changed, and bigger the more it changed. It also catches the model becoming less sure, even when its top guess stays the same.

What comes out: for each chunk, a grid of damage numbers, with one row per damaged token and one column per position checked. Damaging a token can't change predictions made before it, so the left part of each row is always 0. The diagram in the next section shows this grid.

How I know the numbers are right

Before trusting the oracle, I checked it six ways. Every check passed.

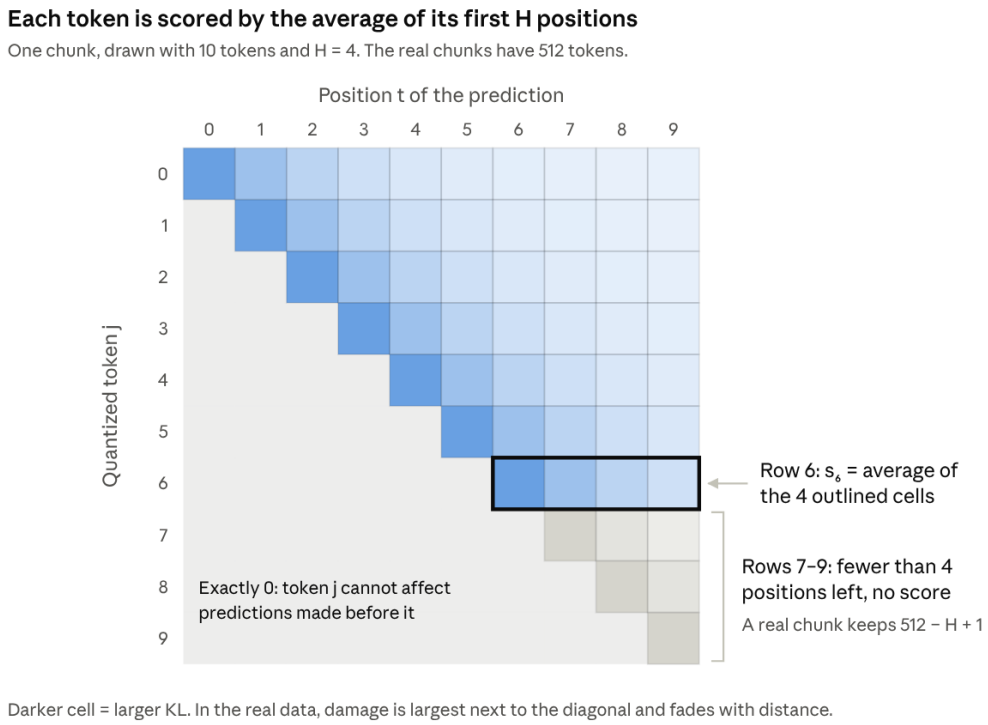
- **Unit tests:** my cache works exactly like the normal one when nothing is damaged.
- **Known result:** Llama-3-8B gets a perplexity of 6.136, matching the published 6.1.
- **No damage, no change:** with nothing quantized, the oracle measures exactly 0.
- **No effect on the past:** damaging a token never changes earlier predictions.
- **Same result every time:** rerunning gives identical numbers.
- **Fewer bits, more damage:** squeezing every token of Llama-3-8B gives the results below.

Bits per number	Perplexity (lower is better)
16 (no squeezing)	6.136
8	6.137
4	6.344
2	294.5

8 bits is basically free, 4 bits costs about 3%, and 2 bits breaks the model. That's why choosing which tokens to squeeze matters.

Turning the grid into one score per token

Each token's row has up to 512 damage numbers. To compare tokens, I need one number for each. So I take the average of the first H numbers in the row, starting at the token itself. That's the token's sensitivity score.



Darker cell = larger KL. In the real data, damage is largest next to the diagonal and fades with distance.

Schematic of one chunk's KL matrix at toy size (10 tokens, H = 4)

In the diagram, row 6's score is the average of its 4 outlined cells. Rows 7–9 don't have 4 cells left, so they get no score.

Why a fixed window of H? It's the fairest option. Here's a made-up example where every token does exactly the same damage, so every token should get the same score:

How to score	Token 0	Token 511
Add up the whole row	52.5	1.0
Average the whole row	0.10	1.0
Average a fixed window (H = 4)	0.425	0.425

Adding up the row favors early tokens, because they have more positions after

them. Averaging the whole row favors late tokens. Only the fixed window treats every token the same.

The cost is that the last few tokens of each chunk don't get a score. I tried H = 32, 64 and 128 to make sure the choice doesn't change the results. It doesn't (see below), so H = 64 is the main one.

What I found

Which tokens are most sensitive (H = 64)

Figure 1 shows each token's sensitivity score by position. Higher means damaging that token hurts more. Each gridline up means 10× more damage.

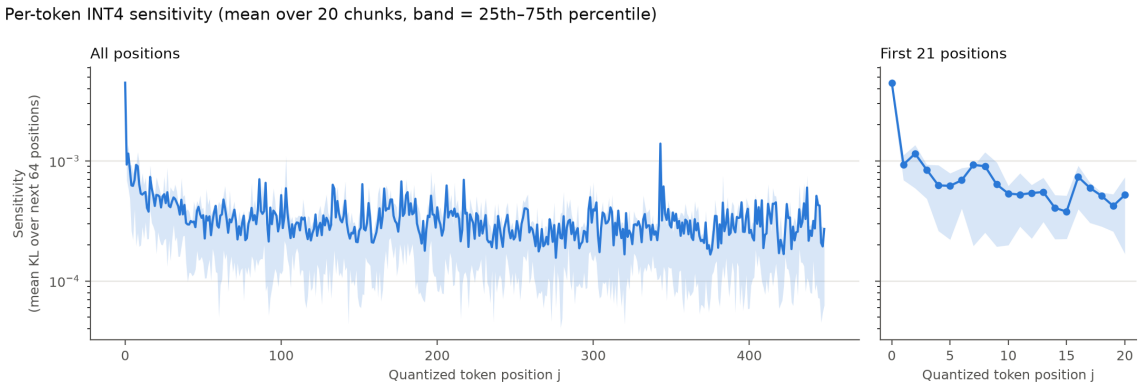


Figure 1. The line is the average over the 20 chunks. The shaded band shows where the middle half of the chunks fall. The right side zooms in on the first 21 tokens.

1. The first token is special. Damaging token 0 hurts about 15× more than a typical token. This is called an attention sink: the model puts a lot of extra attention on the first token, so every later token leans on it. Takeaway: always keep the first token at full precision. Keeping the first 4 is cheap insurance, at under 1% of the cache.

2. The first ~50 tokens matter a little more. After that, it levels off. Early on, the model has only a few tokens to look at, so each one counts more.

3. After that, position tells you very little. Tokens at the same position can differ 3–5× from chunk to chunk. So what a token is matters more than where it is, and that's what the cheap signals need to catch.

4. One big spike, caused by a famous title. At token 343, one chunk has a huge score, about 90× normal. The token is " That" in the album title *Now That's What I Call Music! 46*. Here's how much damaging it hurts each next word:

Next word	Damage (KL)
's	almost none
What	0.27
I	0.07
Call	0.95
Music	0.16
After the title	almost none

The model uses ” That” to recognize the title and guess the next words. Blur it, and the model is much less sure of them. Once the title ends, the damage disappears. So a token can matter because of what comes after it, not where it sits.

Does my choice of H matter?

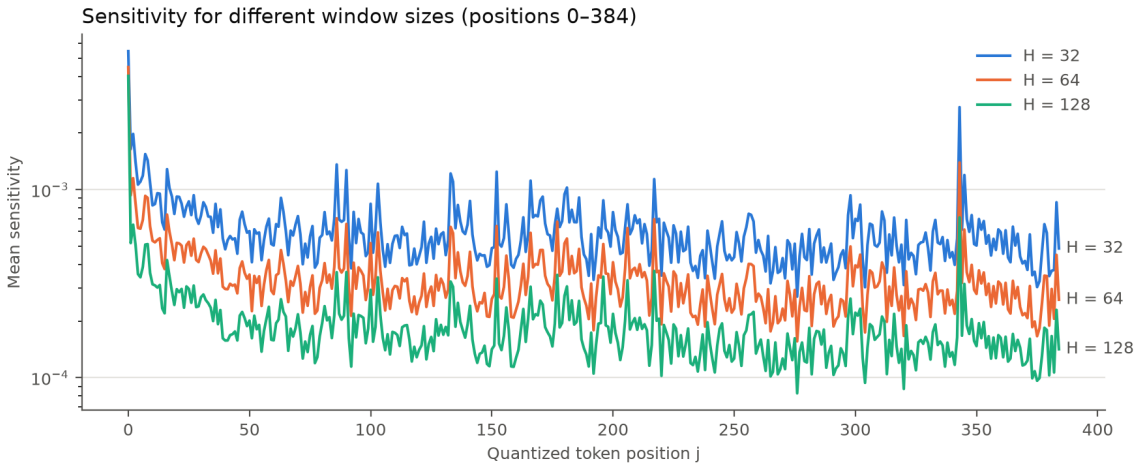


Figure 2. Average score for the three window sizes, over the positions all three share (0–384).

No. A bigger window gives lower scores overall, because damage fades the farther you get from the damaged token. But the shape stays the same, and the same tokens come out on top:

Comparison	Match score (Kendall)	Same top 10%
H = 32 vs. 64	0.92	90%
H = 64 vs. 128	0.93	90%
H = 32 vs. 128	0.86	82%

A match score of 0.92 means the two rankings agree on about 96% of token pairs.

"Same top 10%" means about 34 of the 38 most sensitive tokens are the same. So H = 64 is a safe choice.

How the cheap signals did

Raw attention wins. It matches the answer key best. Key norm barely beats random guessing, even when done exactly the way its paper says.

I computed each signal on the same 20 chunks and turned it into one score per token. I also graded two simple guesses, to see whether the signals beat them:

- **Position:** assume earlier tokens are more important.
- **Random:** pick at random.

Signal	Match score (Kendall)	Same top 10% as the oracle
Raw attention (H2O)	0.35	28%
Normalized attention (ZipCache)	0.24	19%
Position (simple guess)	0.11	25%
Key norm (Devoto et al.)	0.07	15%
Random (simple guess)	0.00	10%

A match score of 1 would mean the same ranking as the oracle, and 0 means no better than random.

- **Raw attention ranks tokens best,** far better than position. Even its worst chunk beats position's best chunk.
- **But on the top 10%, it barely beats position** (28% vs. 25%). The top 10% is what the cache actually protects, so there's room for a better signal.
- **Normalized attention does worse than raw.** It removes the early-token advantage, but early tokens really do matter a little more, so it throws away useful information.
- **Key norm barely beats random.** Skipping layers 0 and 1, like the paper does, gives the same result.
- **The first token isn't what's driving this.** Removing it changes the numbers by 0.015 or less.

Why key norm does so badly

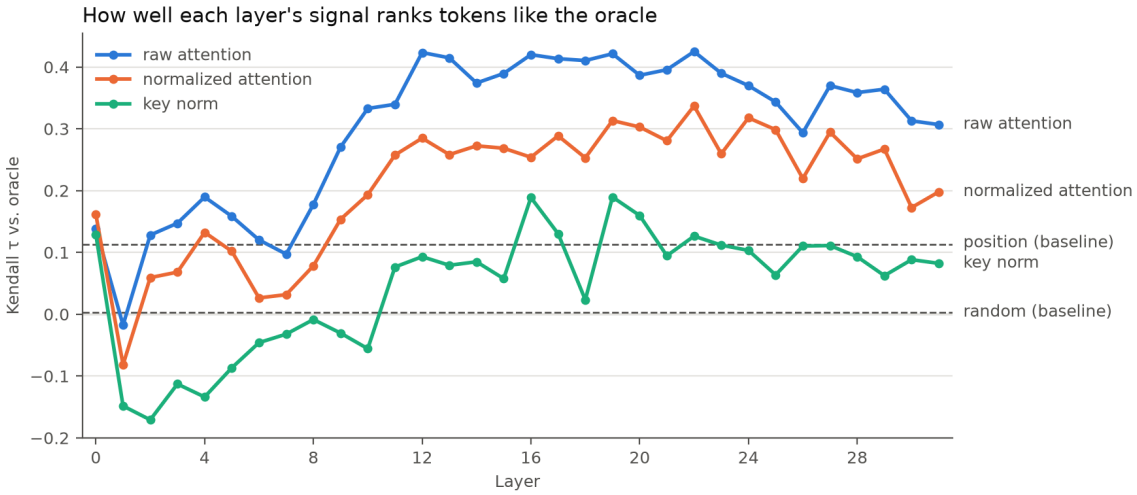


Figure 3. The match score for each layer on its own. The dashed lines are the two simple guesses.

Key norm flips direction partway through the model. In layers 1–10, its score is below zero: there, bigger keys go with more important tokens, the opposite of what the paper says. From layer 11 on, it points the right way. Averaged together, the two halves cancel out.

Why doesn't it work the way the paper says? My best guess: the paper used key norm to decide which tokens to throw away, not which to squeeze. When you squeeze a token, its rounding error matters too, and tokens with small keys also get small rounding errors. The two effects may cancel. I haven't tested this yet.

The plot also shows that raw attention works best in the middle layers, around layers 12–22.

What this means for the project: raw attention is the signal to use. Next, I'll test it inside the cache. I'll keep its top tokens at full precision, squeeze the rest, and compare quality against squeezing everything, protecting the earliest tokens, and protecting the oracle's own top tokens.

Extra

These results come from one small model, one text and one bit-width, so I am treating them as a first look, not a final solution.

What this study doesn't cover yet:

- **Only one setup:** one small model, 20 chunks, one text, and only 4-bit damage.

- **Keys and values together:** I damaged both at once, in every layer, so I can't tell which part causes the damage.
- **No recent-token window:** our plan keeps the newest 128 tokens at full precision, but the oracle damages a token right away. So it measures damage the real plan would partly avoid.
- **Small samples:** each position's average comes from only 20 chunks, so one unusual chunk (like the " That" one) can pull it up.
- **Attention looks ahead:** the attention signals use the whole chunk, including tokens that come later. A real cache generating text wouldn't have those yet. Key norm doesn't have this problem.

Technical details

The exact methods, for anyone who wants them.

- **Chunks:** the WikiText-2 (raw) test set is joined into one text, tokenized, and cut into back-to-back 512-token chunks. The first 20 are used.
- **Predictions:** the model's logits are turned into log-probabilities with log-softmax, which stays accurate even for tiny probabilities.
- **Quantization:** asymmetric min-max, per token and per KV head, with one group over the full head dimension (64). Each vector is rounded to 16 evenly spaced levels between its own min and max, then converted straight back to floats ("fake quantization"). It's applied inside the KV cache, so attention in the same layer already sees the damaged values. Keys and values are both quantized, in all 32 layers.
- **Damage spreads:** once token j's keys and values are off in one layer, later tokens' hidden states shift slightly, and that carries into the layers above.
- **KL divergence** at position t when token j is quantized, with P as the clean prediction and Q as the damaged one:

$$\mathrm{KL}_{j,t} = \sum_{v \in \mathrm{vocab}} P_t(v)\big(\log P_t(v) - \log Q_{j,t}(v)\big)$$

- **Sensitivity score** for token j, with window H:

$$s_j = \frac{1}{H} \sum_{t=j}^{j+H-1} \mathrm{KL}_{j,t}$$

- **Grading:** every comparison is computed per chunk, then averaged over the 20 chunks. For Kendall , (1 +) / 2 is the chance that the two rankings agree on which of two tokens matters more. Spearman gave the same conclusions everywhere: 0.96–0.99 between window sizes, and 0.50 (raw attention), 0.34 (normalized), 0.17 (position), 0.11 (key norm) and 0.01

(random) for the signals. The top-10% check uses the top 38 tokens for the window comparisons (385 positions) and the top 44 for the signals (449 positions).

- **Signals:** computed from one forward pass with eager attention, averaged over heads and over all 32 layers. Key norm's sign is flipped so that higher always means more important. Key norm averaged over layers 2–31 only (the paper's choice) scored 0.07, the same as all layers. Picking the best layers from Figure 3 and then grading on the same chunks would overstate the result, so the main table uses the plain average.
- **Simple guesses:** position scores each token as minus its position. Random uses uniform random scores with a fixed seed.